RÉCAPITULATIF

API REST – EXPRESS – MONGODB – ASYNCHRONE – JWT

Thème : Gestion des tâches (Tasks)

INITIATION À EXPRESS (BASE)

Définition

Express est un framework Node.js qui permet de créer facilement un serveur web et des APIs.

Installation

```
npm init -y
npm install express
```

Exemple simple serveur

```
const express = require("express");
// importer le module express
const app = express();
// créer une application express
app.use(express.json());
// permet de lire les données JSON envoyées par le client

//anciens versions express

// npm install express body-parser
// const bodyParser = require("body-parser");
//app.use(bodyParser.json());

app.listen(3000, () => {
  // démarrage du serveur sur le port 3000
  console.log("Serveur lancé sur http://localhost:3000");
  // message affiché si serveur fonctionne
});
```

API REST (ROUTES)

Définition

Une API REST permet de communiquer avec une application via HTTP.

Méthode	Rôle
GET	Lire
POST	Ajouter
PUT	Modifier
DELETE	Supprimer

Exemple (Tasks)

```
app.get("/tasks", (req, res) => {
  // route pour récupérer les tâches
  res.send("Liste des tâches");
  // envoyer réponse au client
});
app.post("/tasks", (req, res) => {
  // route pour ajouter une tâche
  res.send("Tâche ajoutée");
  // confirmation ajout
});
```

CALLBACK vs PROMISE vs ASYNC/AWAIT

CALLBACK

```
const fs = require("fs");
// importer module fichier
fs.readFile("tasks.json", (err, data) => {
  // lire fichier de façon asynchrone
  if (err) {
    // si erreur
```

```
console.log("Erreur :", err);
    // afficher erreur
  } else {
    // si succès
    console.log(data.toString());
    // afficher contenu fichier
  }
});
```

PROMISE (resolve / reject)

```
function operation() {
  return new Promise((resolve, reject) => {
    // création d'une promesse
    let success = true;
    // condition simulée
    if (success) {
      resolve("Tâche réussie");
      // succès
    } else {
      reject("Erreur dans la tâche");
      // échec
    }
  });
}
```

ASYNC / AWAIT

```
async function test() {
  // fonction asynchrone
  try {
    let result = await operation();
    // attendre résultat

  console.log(result);
    // afficher résultat
```

```
  } catch (err) {
    // gestion erreur
    console.log(err);
    // afficher erreur
  }
}
```

MONGODB + MONGOOSE

Définition

MongoDB = base de données NoSQL
Mongoose = outil pour gérer MongoDB

Connexion

```
const mongoose = require("mongoose");
// importer mongoose
mongoose.connect("mongodb://127.0.0.1:27017/tasksdb")
  // connexion base tasksdb
  .then(() => console.log("MongoDB connecté"))
  // succès connexion
  .catch(err => console.log(err));
  // erreur connexion
```

Modèle Task

```
const taskSchema = new mongoose.Schema({
  title: String,
  // titre de la tâche

  completed: Boolean
  // statut tâche (true/false)
});
const Task = mongoose.model("Task", taskSchema);
// création modèle Task
```

CRUD

CRUD SUR TABLEAU / JSON (SANS DB)

```
let tasks = [];
// tableau en mémoire
// CREATE
tasks.push({ id: 1, title: "Task 1" });
// READ
console.log(tasks);
// UPDATE
tasks[0].title = "Task modifiée";
// DELETE
tasks = tasks.filter(t => t.id !== 1);
```

✓CRUD simple sans base de données

✓utilisé avec fichiers JSON

CRUD MONGODB (NO SQL)

CREATE

```
app.post("/tasks", async (req, res) => {
  // route ajout tâche
  const task = await Task.create(req.body);

  // créer tâche dans MongoDB

  res.json(task);
  // envoyer résultat
});
```

READ

```
app.get("/tasks", async (req, res) => {
  // récupérer toutes les tâches
  const tasks = await Task.find();
  // lire toutes tâches
  res.json(tasks);
  // envoyer résultat
});
```

UPDATE

```
app.put("/tasks/:id", async (req, res) => {
  // modifier tâche par ID
  const task = await Task.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true }
    // retourne version modifiée
  );
  res.json(task);
  // envoyer tâche modifiée
});
```

DELETE

```
app.delete("/tasks/:id", async (req, res) => {
  // supprimer tâche par ID
  await Task.findByIdAndDelete(req.params.id);
  // suppression
  res.send("Tâche supprimée");
  // confirmation
});
```

JWT – AUTHENTIFICATION

Définition

JWT = système de token sécurisé

Header :

Authorization: Bearer TOKEN

REGISTER

```
app.post("/register", async (req, res) => {
  // route inscription
  const hashed = await bcrypt.hash(req.body.password, 10);
  // crypter mot de passe
  const user = await User.create({
    email: req.body.email,
```

```
  password: hashed
 });
 // créer utilisateur
 res.json(user);
 // réponse
});
```

LOGIN

```
app.post("/login", async (req, res) => {
 // route connexion
 const user = await User.findOne({ email: req.body.email });
 // chercher utilisateur
 if (!user) return res.send("Email incorrect");
 // si email invalide
 const valid = await bcrypt.compare(req.body.password, user.password);
 // vérifier mot de passe
 if (!valid) return res.send("Mot de passe incorrect");
 // si faux
 const token = jwt.sign(
  { id: user._id, email: user.email },
  "SECRET_KEY",
  { expiresIn: "1h" }
 );
 // créer token
 res.json({ token });
});
```

MIDDLEWARE

```
function verifyToken(req, res, next) {
 // fonction protection
 const header = req.headers["authorization"];
 // récupérer token
 if (!header) {
  return res.status(401).send("Token requis");
```

```
 }
 // si pas de token

 const token = header.split(" ")[1];
 // enlever Bearer
 jwt.verify(token, "SECRET_KEY", (err, decoded) => {
   // vérifier token

 if (err) {
     return res.status(403).send("Token invalide");
   }
   // erreur token
   req.user = decoded;
   // stocker utilisateur
   next();
   // continuer
 });
}
```

ROUTE PROTÉGÉE

```
app.get("/tasks", verifyToken, async (req, res) => {
 // route protégée
 const tasks = await Task.find();
 // récupérer tâches
 res.json(tasks);
 // envoyer résultat
});
```